

Organizing a C/C++ Project with a Makefile

Course Document

The src and include convention, built with a Makefile

This document shows an alternative way to organize a C or C++ project: splitting header files into their own include folder, separate from the src folder that holds your source files, and building the project with a Makefile instead of relying on an IDE to compile it for you.

A note before you start: This layout is not required for this course's assignments unless stated otherwise. If the plain src-only layout described in *Organizing a C/C++ Project* already works for you, you do not need to switch. Use this document if you would rather work the way many real C and C++ projects are set up outside of a course.

1. The Project Layout

The example below reorganizes the same `lastlabw01` project from *Organizing a C/C++ Project*. Header files move into an `include` folder, and source files stay in `src`. Splitting them this way keeps a module's public interface, its header, visible on its own, separate from the implementation details in the matching `.cpp` file.

```
lastlabw01/  
  include/  
    driver/  
      Driver.h  
    vehicles/  
      Vehicle.h  
  src/  
    driver/  
      Driver.cpp  
    vehicles/  
      Vehicle.cpp  
  main.cpp  
  Makefile
```

Table 1. What each folder or file is for.

Location	What It Is
<code>lastlabw01/</code>	The top-level project folder.
<code>lastlabw01/include/</code>	Holds header files only, one subfolder per module.
<code>lastlabw01/include/driver/ Driver.h</code>	The public interface for the driver module.
<code>lastlabw01/src/</code>	Holds source files, plus <code>main.cpp</code> .
<code>lastlabw01/src/driver/Driver.cpp</code>	The implementation of <code>Driver.h</code> .

Location	What It Is
lastlabw01/src/main.cpp	The entry point of the program. It is not part of any module, so it sits directly inside src.
lastlabw01/Makefile	The build instructions for the whole project, described below.

2. The Makefile

Save the following as a file named exactly `Makefile`, with no file extension, in the project's top-level folder, next to `include` and `src`.

```
CXX = g++
CXXFLAGS = -Wall -std=c++17 -Iinclude

SRCS = src/main.cpp src/driver/Driver.cpp src/vehicles/Vehicle.cpp
OBJS = $(SRCS:.cpp=.o)
TARGET = lastlabw01

$(TARGET): $(OBJS)
    $(CXX) $(CXXFLAGS) -o $(TARGET) $(OBJS)

clean:
    rm -f $(OBJS) $(TARGET)

.PHONY: clean
```

Compiler settings: `CXX` names the compiler to use, and `CXXFLAGS` lists the options passed every time. The `-Iinclude` flag tells the compiler to look inside `include` whenever a file writes `#include "Driver.h"`, which is why the source files do not need a longer relative path to reach their headers.

Source and object lists: `SRCS` lists every `.cpp` file in the project. `OBJS` takes that same list and swaps each `.cpp` for a `.o`, the compiled but not yet linked form of each file.

The build rule: The line `lastlabw01: $(OBJS)` says the program depends on those object files. Running `make` compiles whichever `.cpp` files changed into `.o` files, then links them together into the program named `lastlabw01`.

clean: A second target that deletes everything `make` produced, so a fresh build can start from nothing. `.PHONY: clean` tells `make` that `clean` is an action to run, not a file it should look for.

3. Building, Running, and Cleaning

From a terminal inside the project folder, run:

```
make
./lastlabw01
make clean
```

Typing `make` alone builds the first target listed in the file, which is `lastlabw01`. `make clean` removes the compiled program and its object files.

Using this layout in Eclipse CDT: Create a Makefile Project instead of a Managed Build project (File, New, C/C++ Project, Makefile Project), and Eclipse will call this Makefile instead of compiling the project automatically.